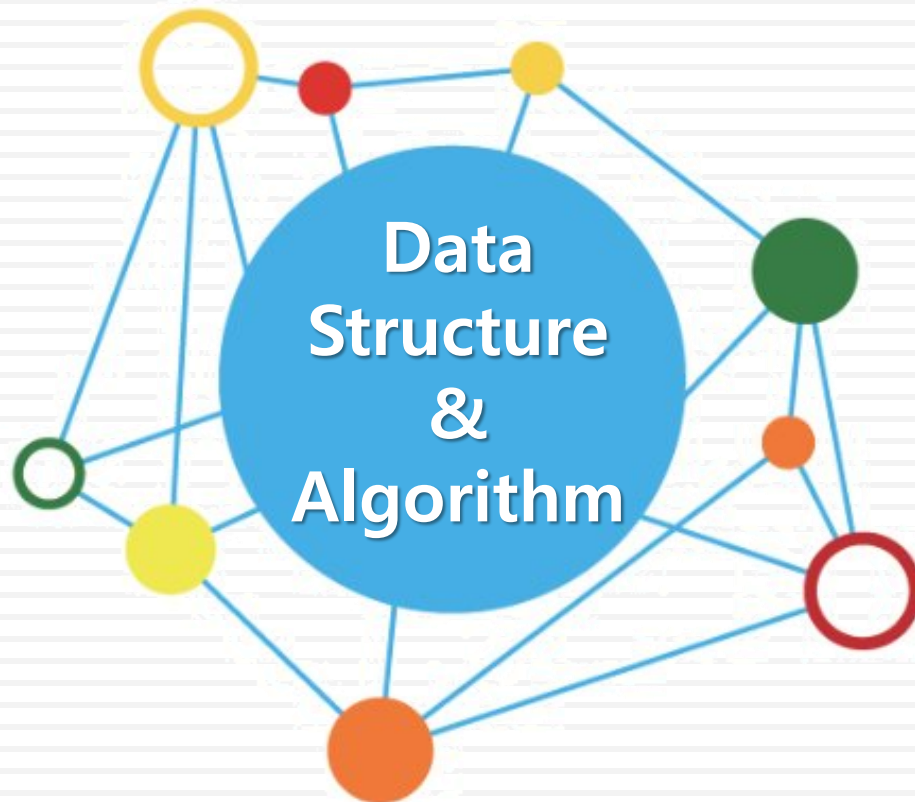


데이터 구조론



유길현

그래프(Graph)

❖ 너비 우선 탐색(breadth first search : BFS)

■ 순회 방법

- 시작 정점으로부터 인접한 정점들을 모두 차례로 방문하고 나서, 방문했던 정점을 시작으로 하여 다시 인접한 정점들을 차례로 방문하는 방식
- 가까운 정점들을 먼저 방문하고 멀리 있는 정점들은 나중에 방문하는 순회 방법
- 인접한 정점들에 대해 차례로 다시 너비 우선 탐색을 반복해야 하므로 선입 선출의 구조를 갖는 큐를 사용

❖ 너비 우선 탐색(breadth first search : BFS)

■ 탐색 순서

- ① 시작 정점 v 를 결정하여 방문한다.
- ② 정점 v 에 인접한 정점 중에서 방문하지 않은 정점을 차례로 방문하여 큐에 enqueue한다.
- ③ 방문하지 않은 인접한 정점이 없으면 방문했던 정점에서 인접한 정점을 다시 차례로 방문하기 위해 큐에서 dequeue하여 받은 정점을 v 로 설정하고 ②를 반복한다.
- ④ 큐가 공백이 될 때 까지 ②~③을 반복한다.

❖ 너비 우선 탐색

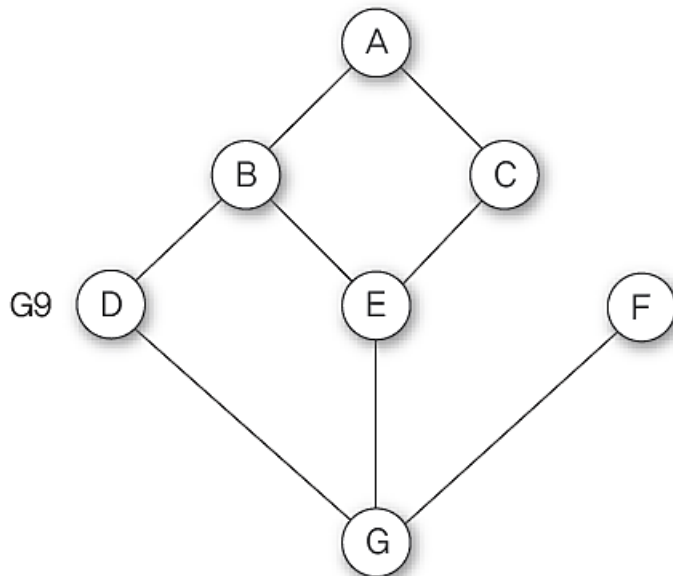
알고리즘

그래프의 너비 우선 탐색

```
BFS(v)
  for (i ← 0; i < n; i ← i + 1) do {
    visited[i] ← false;
  }
  Q ← createQueue();
  visited[v] ← true;
  v 방문;
  while (not isEmpty(Q)) do {
    while (visited[v의 인접정점 w] = false) do {
      visited[w] ← true;
      w 방문;
      enqueue(Q, w);
    }
    v ← dequeue(Q);
  }
end BFS()
```

❖ 너비 우선 탐색

- 알고리즘에 따른 그래프 G9의 너비 우선 탐색 과정
 - 초기 상태 : 배열 visited를 False로 초기화, 공백 큐를 생성



정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
	F	F	F	F	F	F	F

visited

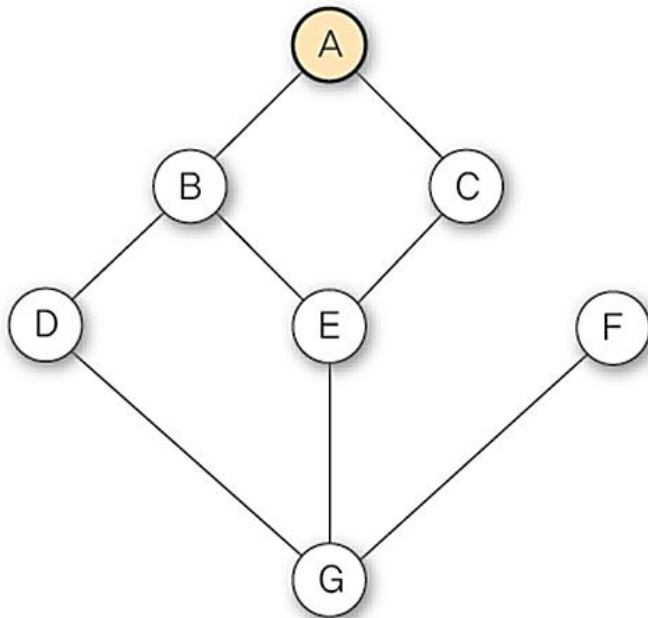
Q



❖ 너비 우선 탐색

① 정점 A를 시작으로 너비 우선 탐색을 시작한다.

visited[A] ← true;
A 방문;



정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
visited	T	F	F	F	F	F	F

visited

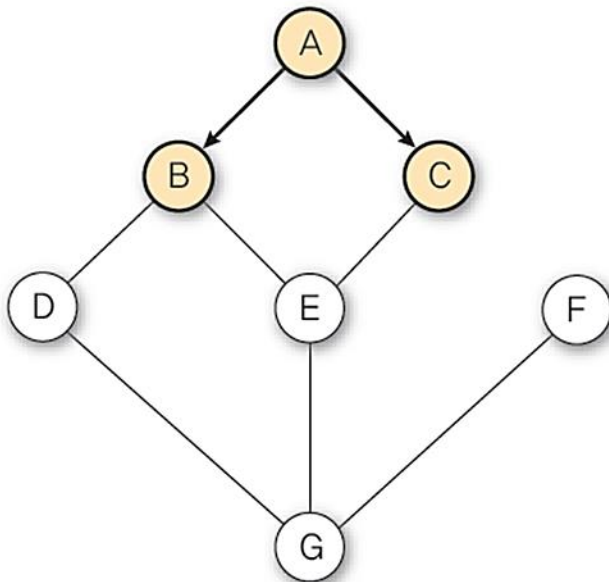
Q



❖ 너비 우선 탐색

- ② 정점 A에서 방문하지 않은 모든 인접 정점 B, C를 방문하고 큐에 enqueue 한다.

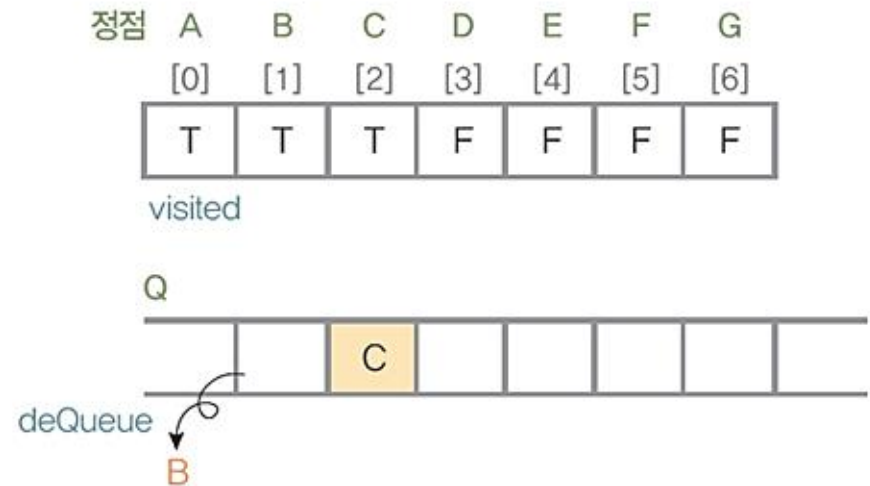
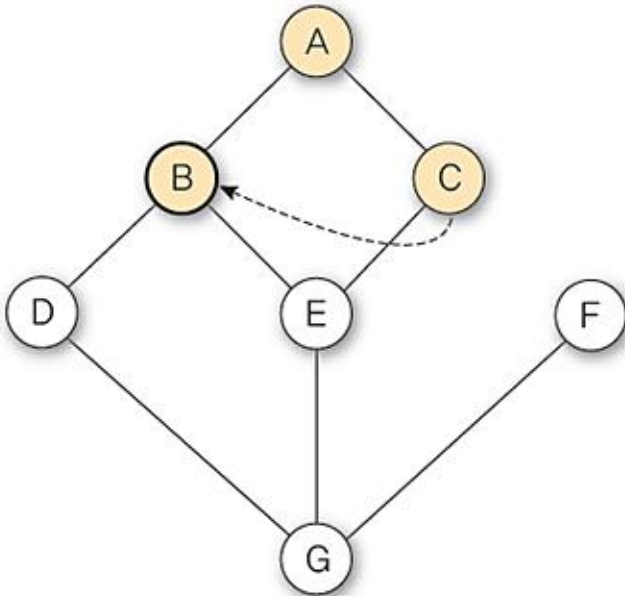
```
visited[(A의 방문 안한 인접정점 B와 C)] ← true;  
(A의 방문 안한 인접정점 B와 C) 방문;  
enqueue(Q, (A의 방문 안한 인접정점 B와 C));
```



❖ 너비 우선 탐색

- ③ 정점 A에 대한 인접 정점들을 처리했으므로, 너비 우선 탐색을 계속할 다음 정점을 찾기 위해 큐를 deQueue하여 B를 받음

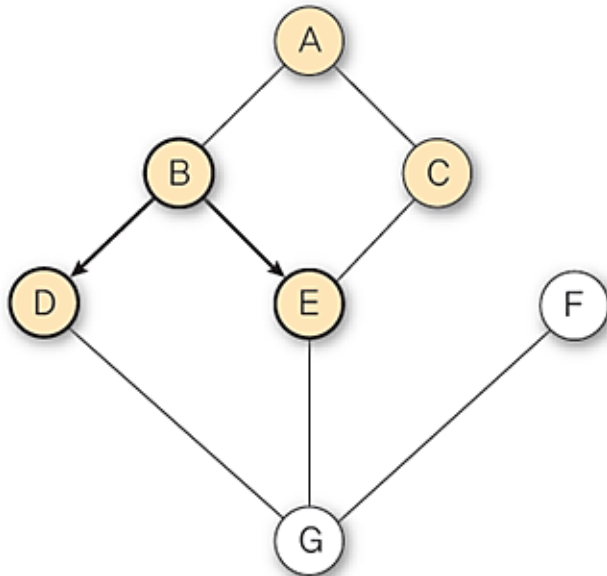
```
v ← deQueue(Q);
```



❖ 너비 우선 탐색

- ④ 정점 B에서 방문하지 않은 모든 인접 정점 D, E를 방문하고 큐에 enqueue

```
visited[(B의 방문 안한 인접정점 D와 E)] ← true;  
(B의 방문 안한 인접정점 D와 E) 방문;  
enqueue(Q, (B의 방문 안한 인접정점 D와 E));
```



B의 인접 정점

정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
visited	T	T	T	T	T	F	F

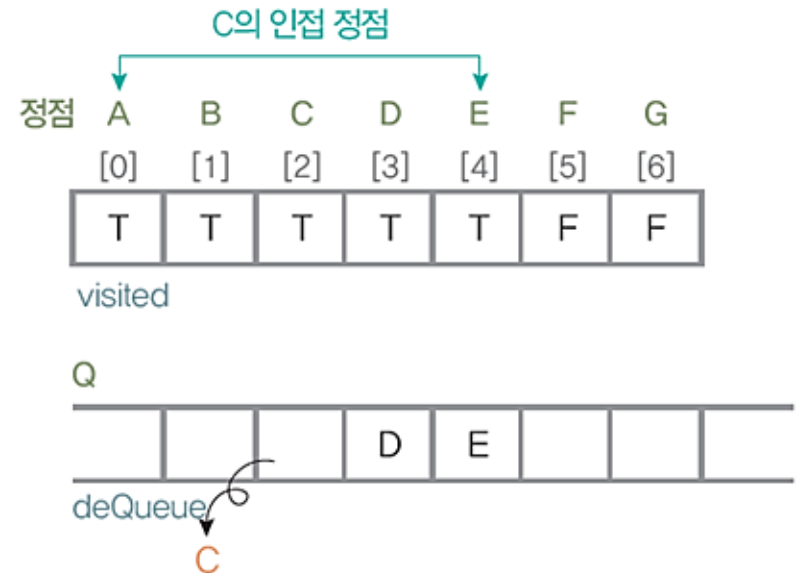
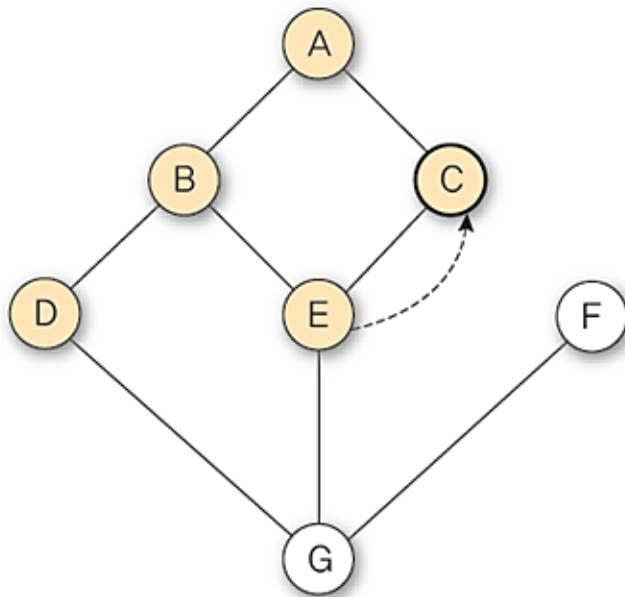
Q

		C	D	E			
--	--	---	---	---	--	--	--

❖ 너비 우선 탐색

- ⑤ 정점 B에 대한 인접 정점들을 처리했으므로, 너비 우선 탐색을 계속할 다음 정점을 찾기 위해 큐를 deQueue하여 C를 받음

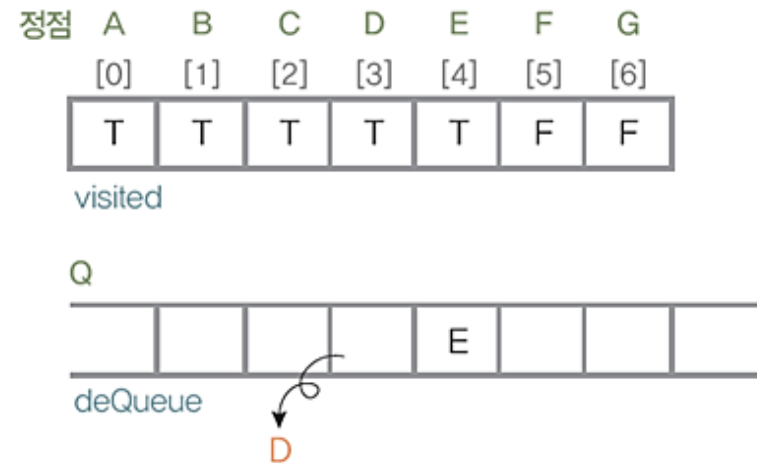
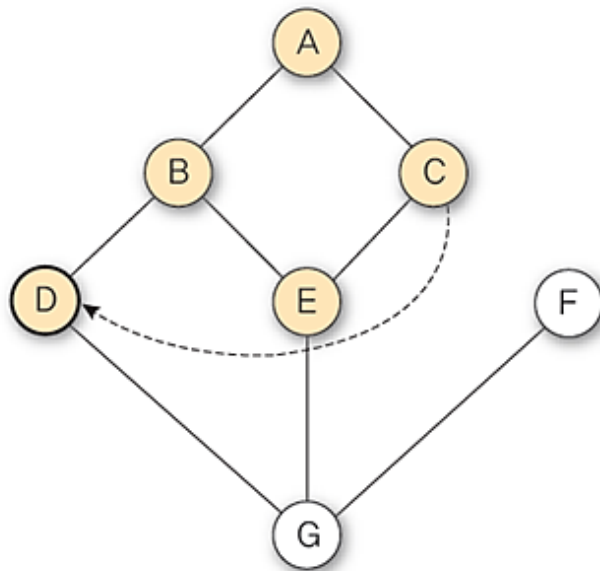
$v \leftarrow \text{deQueue}(Q);$



❖ 너비 우선 탐색

- ⑥ 정점 C에는 방문하지 않은 인접 정점이 없으므로, 너비 우선 탐색을 계속할 다음 정점을 찾기 위해 큐를 deQueue하여 D를 받음

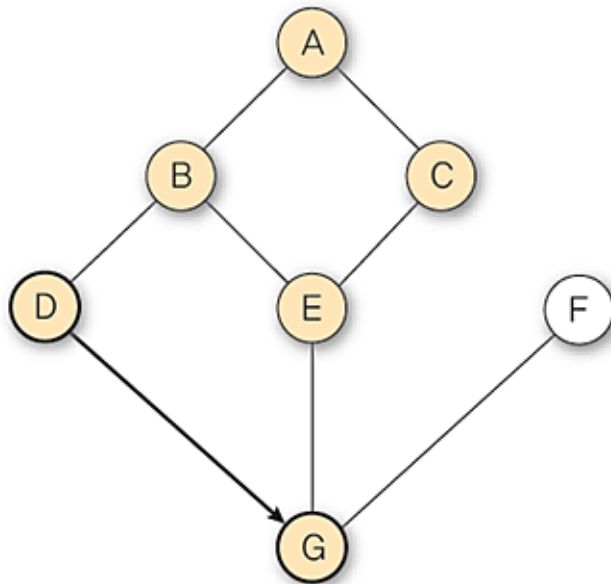
```
v ← deQueue(Q);
```



❖ 너비 우선 탐색

- ⑦ 정점 D에서 방문하지 않은 인접 정점 G를 방문하고 큐에 enqueue

```
visited[(D의 방문 안한 인접정점 G)] ← true;  
(D의 방문 안한 인접정점 G) 방문;  
enqueue(Q, (D의 방문 안한 인접정점 G));
```



D의 인접 정점

정점	A	B	C	D	E	F	G
	[0]	[1]	[2]	[3]	[4]	[5]	[6]
	T	T	T	T	T	F	T

visited

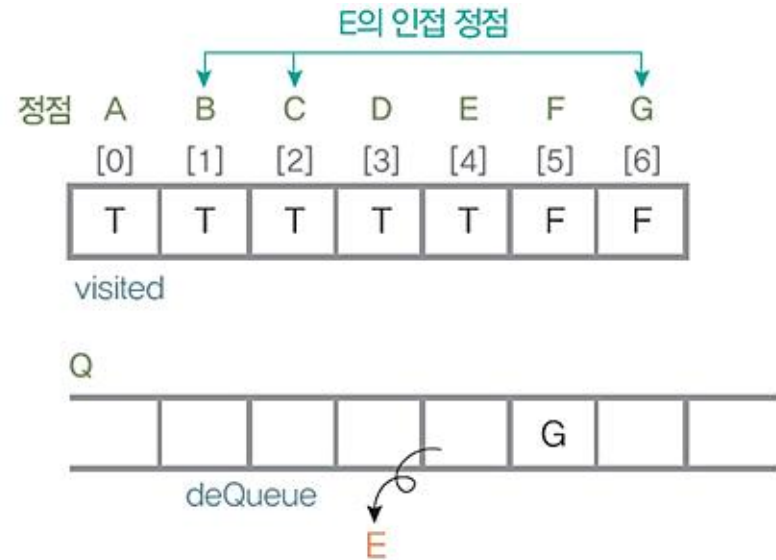
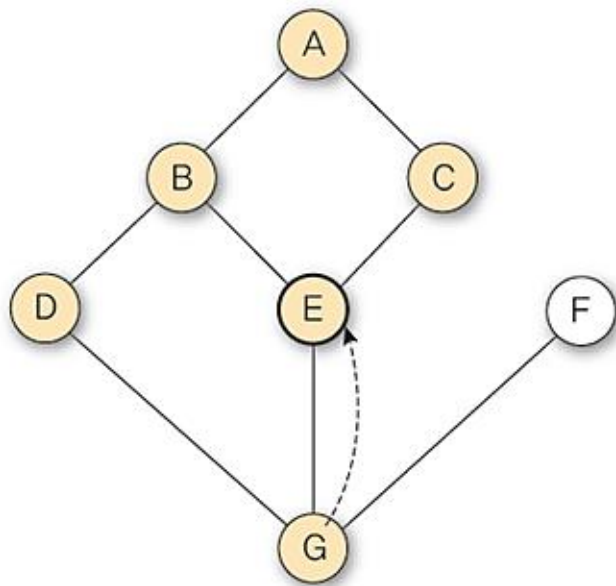
Q

				E	G		
--	--	--	--	---	---	--	--

❖ 너비 우선 탐색

- ⑧ 정점 D에 대한 인접 정점들을 처리했으므로, 너비 우선 탐색을 계속 할 다음 정점을 찾기 위해 큐를 deQueue하여 E를 받음

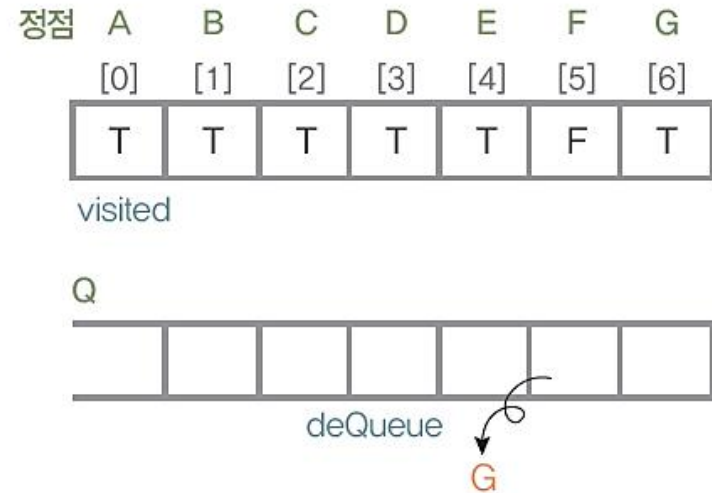
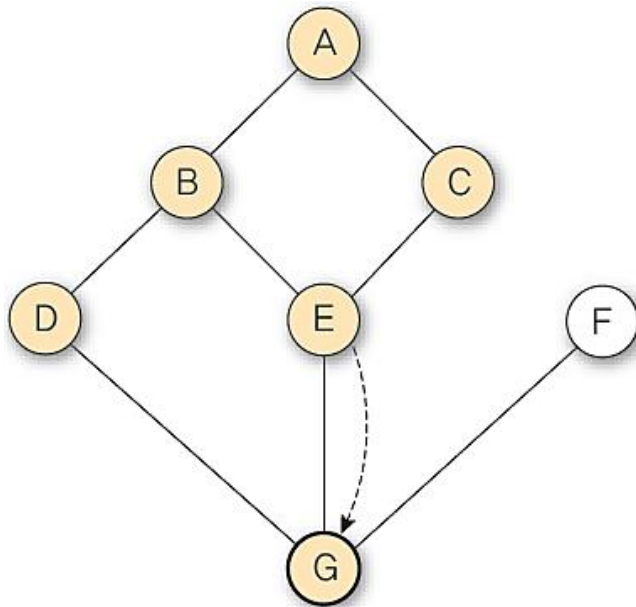
$v \leftarrow \text{deQueue}(Q);$



❖ 너비 우선 탐색

- ⑨ 정점 E에는 방문하지 않은 인접 정점이 없으므로, 너비 우선 탐색을 계속할 다음 정점을 찾기 위해 큐를 deQueue하여 G를 받음

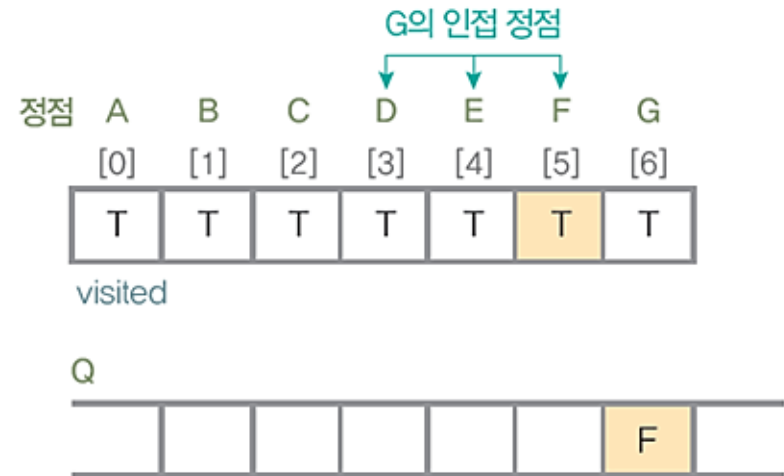
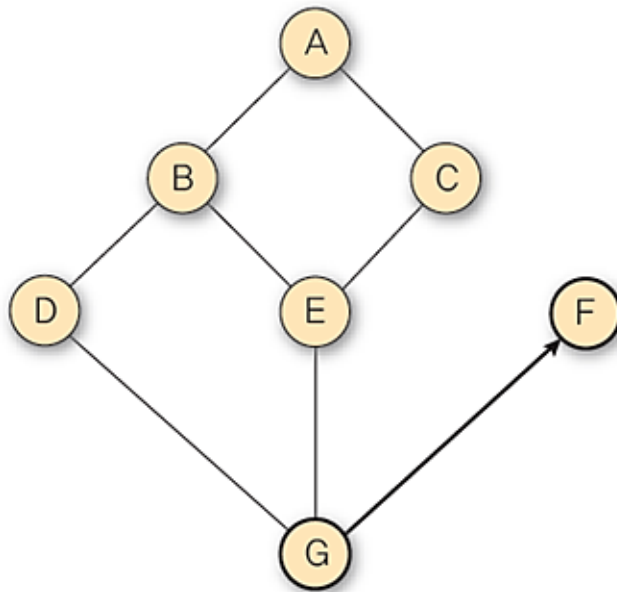
```
v ← deQueue(Q);
```



❖ 너비 우선 탐색

- ⑩ 정점 G에서 방문하지 않은 인접 정점 F를 방문하고 큐에 enqueue

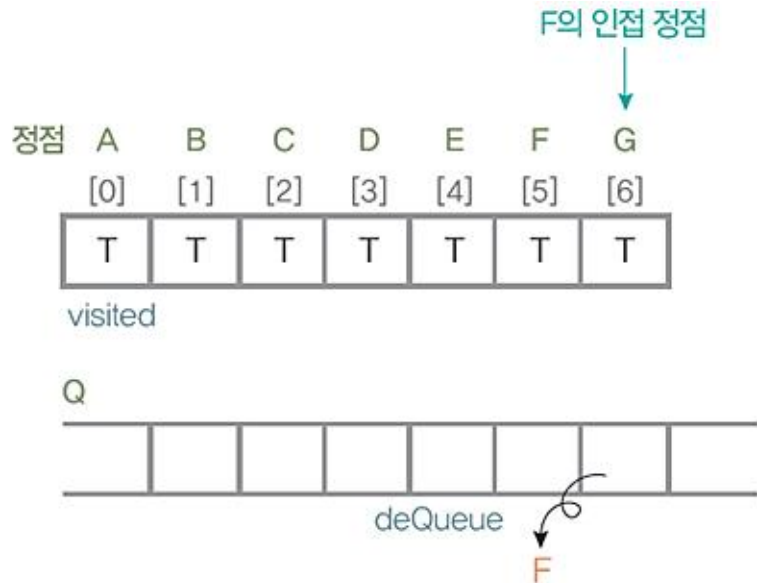
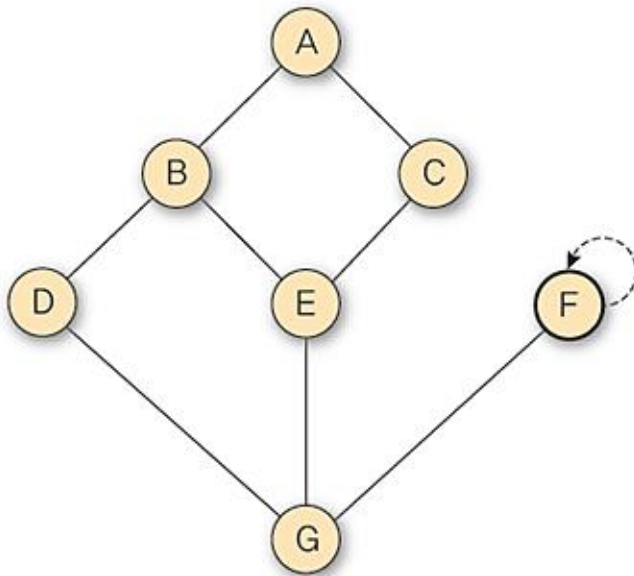
```
visited[(G의 방문 안한 인접정점 F)] ← true;  
(G의 방문 안한 인접정점 F) 방문;  
enqueue(Q, (G의 방문 안한 인접정점 F));
```



❖ 너비 우선 탐색

- ⑪ 정점 G에 대한 인접 정점들을 처리했으므로, 너비 우선 탐색을 계속 할 다음 정점을 찾기 위해 큐를 deQueue하여 F를 받음

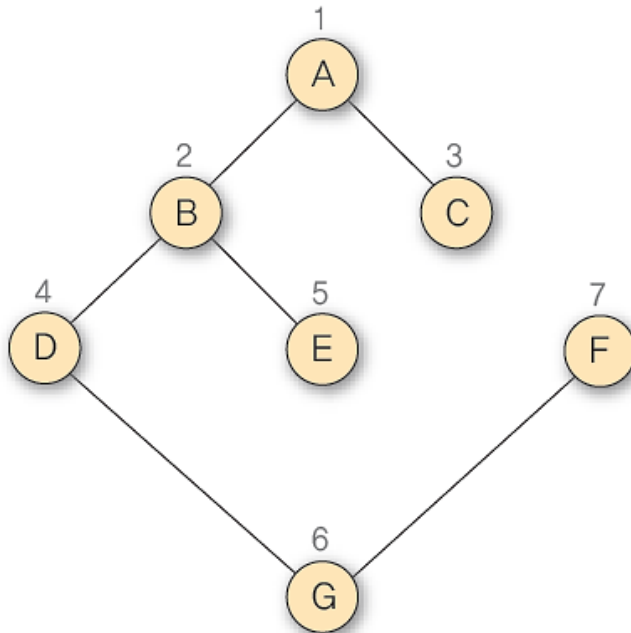
$v \leftarrow \text{deQueue}(Q);$



❖ 너비 우선 탐색

⑫ 정점 F는 모든 인접 정점을 방문 했으므로, 너비 우선 탐색을 계속할 다음 정점을 찾기 위해 큐를 deQueue하는데 큐가 공백이므로 너비 우선 탐색을 종료

- 탐색 경로 : A-B-C-D-E-G-F



❖ 그래프 너비 우선 탐색 프로그램

```
#include <stdio.h>
#include <memory.h>
#include <stdlib.h>
#define MAX_VERTEX 10
#define FALSE 0
#define TRUE 1

// 그래프에 대한 인접 리스트의 노드 구조 정의
typedef struct graphNode {
    int vertex;
    struct graphNode* link;
}graphNode;

typedef struct graphType {
    int n; // 정점 개수
    // 정점에 대한 인접리스트의 헤드 노드 배열
    graphNode* adjList_H[MAX_VERTEX];
    int visited[MAX_VERTEX]; // 정점에 대한 방문 표시를 위한 배열
}graphType;
```

❖ 그래프 너비 우선 탐색하기 프로그램

```
typedef int element;

typedef struct QNode {
    int data;
    struct QNode *link;
} QNode;

typedef struct {
    QNode *front, *rear;
} LQueueType;

LQueueType *createLinkedQueue()
{
    LQueueType *LQ;
    LQ = (LQueueType *)malloc(sizeof(LQueueType));
    LQ->front = NULL;
    LQ->rear = NULL;
    return LQ;
}
```

❖ 그래프 너비 우선 탐색하기 프로그램

```
int isEmpty(LQueueType *LQ)
{
    if (LQ->front == NULL) {
        printf("\n Linked Queue is empty! \n");
        return 1;
    }
    else
        return 0;
}
```

❖ 그래프 너비 우선 탐색하기 프로그램

```
void enqueue(LQueueType *LQ, int item)
{
    QNode *newNode = (QNode *)malloc(sizeof(QNode));
    newNode->data = item;
    newNode->link = NULL;
    if (LQ->front == NULL) {
        LQ->front = newNode;
        LQ->rear = newNode;
    }
    else {
        LQ->rear->link = newNode;
        LQ->rear = newNode;
    }
}
```

❖ 그래프 너비 우선 탐색하기 프로그램

```
int deQueue(LQueueType *LQ)
{
    QNode *old = LQ->front;
    int item;
    if (isEmpty(LQ))
        return 0;
    else {
        item = old->data;
        LQ->front = LQ->front->link;
        if (LQ->front == NULL)
            LQ->rear = NULL;
        free(old);
        return item;
    }
}
```

❖ 그래프 너비 우선 탐색하기 프로그램

```
// 너비 우선 탐색을 위한 초기 공백 그래프를 생성하는 연산
void createGraph(graphType* g)
{
    int v;
    g->n = 0;      // 그래프의 정점 개수를 0으로 초기화
    for (v = 0; v<MAX_VERTEX; v++) {
        // 그래프 정점에 대한 배열 visited를 FALSE로 초기화
        g->visited[v] = FALSE;
        // 인접 리스트에 대한 헤드 노드 배열을 NULL로 초기화
        g->adjList_H[v] = NULL;
    }
}
```

❖ 그래프 너비 우선 탐색하기 프로그램

```
// 그래프 g에 정점 v를 삽입하는 연산
void insertVertex(graphType* g, int v)
{
    if (((g->n) + 1) > MAX_VERTEX) {
        printf("Wn 그래프 정점의 개수를 초과하였습니다!");
        return;
    }
    g->n++;
}
```


❖ 그래프 너비 우선 탐색하기 프로그램

```
// 그래프 g에 간선 (u, v)를 삽입하는 연산
void insertEdge(graphType* g, int u, int v)
{
    graphNode* node;
    if (u >= g->n || v >= g->n) {
        printf("Wn 그래프에 없는 정점입니다!");
        return;
    }
    node = (graphNode *)malloc(sizeof(graphNode));
    node->vertex = v;
    node->link = g->adjList_H[u];
    g->adjList_H[u] = node;
}
```

❖ 그래프 너비 우선 탐색하기 프로그램

```
// 그래프 g에 대한 인접 리스트를 출력하는 연산
void print_adjList(graphType* g)
{
    int i;
    graphNode* p;
    for (i = 0; i < g->n; i++) {
        printf("WnWtWt정점 %c의 인접 리스트", i + 65);
        p = g->adjList_H[i];
        while (p) {
            printf(" -> %c", p->vertex + 65);
            p = p->link;
        }
    }
}
```

❖ 그래프 너비 우선 탐색하기 프로그램

```
// 그래프 g에서 정점 v에 대한 너비 우선 탐색 연산
void BFS_adjList(graphType* g, int v)
{
    graphNode* w;
    LQueueType* Q;           // 큐
    Q = createLinkedQueue(); // 큐 생성
    g->visited[v] = TRUE;    // 시작 정점 v에 대한 배열 값을 TRUE로 설정
    printf(" %c", v + 65);
    enqueue(Q, v);          // 현재 정점 v를 큐에 enqueue
    while (!isEmpty(Q)) {   // 큐가 공백인 아닌 동안 너비 우선 탐색 수행
        v = dequeue(Q);
        // 현재 정점 w를 방문하지 않은 경우
        for (w = g->adjList_H[v]; w; w = w->link) // 인접 정점이 있는 동안 수행
            if (!g->visited[w->vertex]) { // 정점 w가 방문하지 않은 정점인 경우
                g->visited[w->vertex] = TRUE;
                printf(" %c", w->vertex + 65); // 정점 0~6을 A~G로 바꾸어 출력
                enqueue(Q, w->vertex);
            }
    }
    // 큐가 공백이면 너비 우선 탐색 종료
}
```

❖ 그래프 너비 우선 탐색하기 프로그램

```
void main()
{
    int i;
    graphType *G9;
    G9 = (graphType *)malloc(sizeof(graphType));
    createGraph(G9);

    // 그래프 G9 구성
    for (i = 0; i < 7; i++)
        insertVertex(G9, i);
    insertEdge(G9, 0, 2);
    insertEdge(G9, 0, 1);
    insertEdge(G9, 1, 4);
    insertEdge(G9, 1, 3);
    insertEdge(G9, 1, 0);
    insertEdge(G9, 2, 4);
    insertEdge(G9, 2, 0);
    insertEdge(G9, 3, 6);
    insertEdge(G9, 3, 1);
```

❖ 그래프 너비 우선 탐색하기 프로그램

```
insertEdge(G9, 4, 6);
insertEdge(G9, 4, 2);
insertEdge(G9, 4, 1);
insertEdge(G9, 5, 6);
insertEdge(G9, 6, 5);
insertEdge(G9, 6, 4);
insertEdge(G9, 6, 3);
printf("\n G9의 인접 리스트 ");
print_adjList(G9);

printf("\n\n////////////////\n\n너비 우선 탐색 >> ");
BFS_adjList(G9, 0);    // 0번 정점인 정점 A에서 너비 우선 탐색 시작

getchar();
}
```

❖ 그래프 너비 우선 탐색하기 프로그램 : 교재 444p

■ 실행결과

```
C:\WINDOWS\system32\cmd.exe

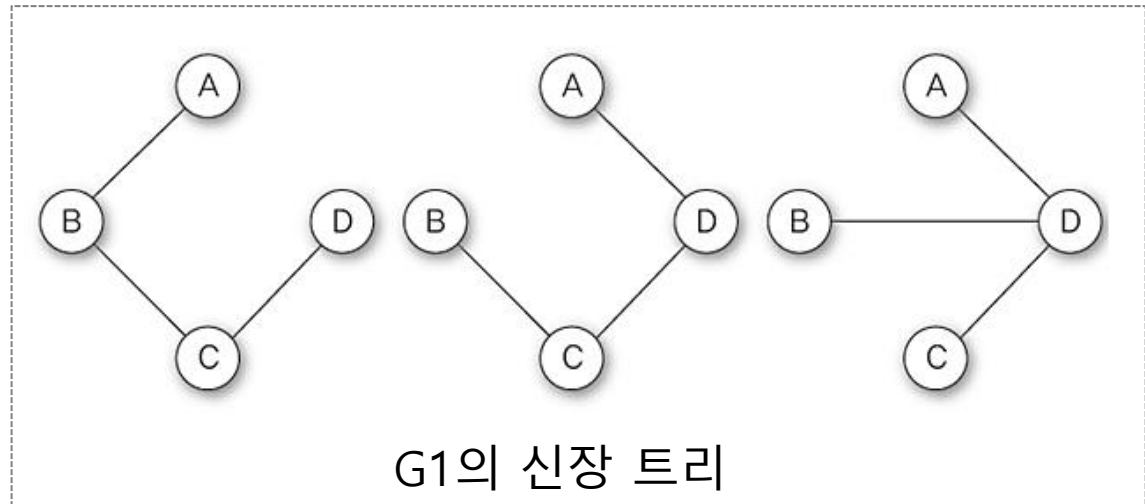
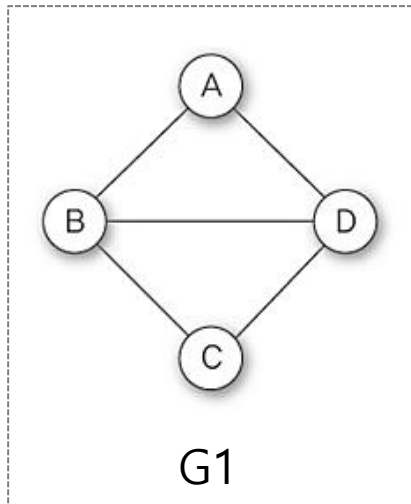
G9의 인접 리스트
정점 A의 인접 리스트 -> B -> C
정점 B의 인접 리스트 -> A -> D -> E
정점 C의 인접 리스트 -> A -> E
정점 D의 인접 리스트 -> B -> G
정점 E의 인접 리스트 -> B -> C -> G
정점 F의 인접 리스트 -> G
정점 G의 인접 리스트 -> D -> E -> F

//////////

너비 우선 탐색 >> A B C D E G F
Linked Queue is empty!
```

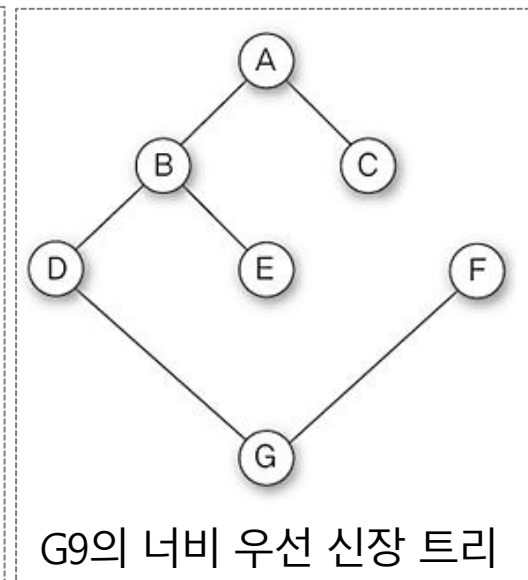
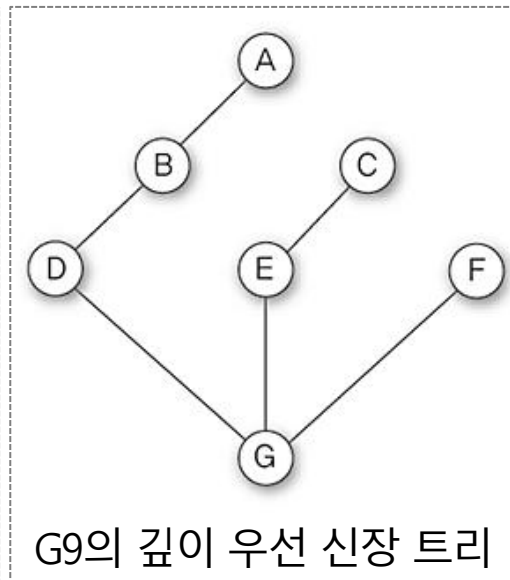
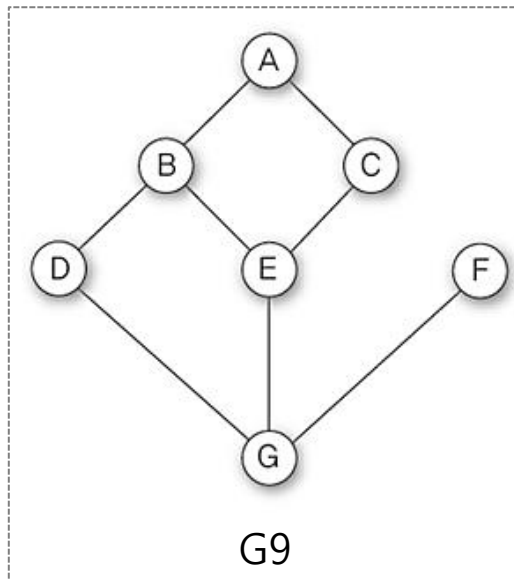
❖ 신장 트리(spanning tree)

- n 개의 정점으로 이루어진 무방향 그래프 G 에서 n 개의 모든 정점과 $n-1$ 개의 간선으로 만들어진 트리
- 신장 트리에서는 사이클이 포함되면 안됨
- 최소 개수의 간선을 이용하여 네트워크를 구성
- 그래프 G_1 과 신장 트리의 예



❖ 신장 트리(spanning tree)

- 깊이 우선 신장 트리(Depth First Spanning Tree)
 - 깊이 우선 탐색을 이용하여 생성된 신장 트리
- 너비 우선 신장 트리(Breadth First Spanning Tree)
 - 너비 우선 탐색을 이용하여 생성된 신장 트리
- 최소 개수의 간선을 이용하여 네트워크를 구성
- 그래프 G9의 신장 트리 예



❖ 최소 비용 신장 트리(Minimum Cost Spanning Tree)

- 무방향 가중치 그래프에서 신장 트리를 구성하는 간선들의 가중치 합이 최소인 신장 트리
 - 가중치 그래프의 간선에 주어진 가중치
 - ⇒ 비용이나 거리, 시간을 의미하는 값
- 응용 예

도로 건설	도시들을 모두 연결하면서 도로의 길이가 최소가 되도록 하는 문제
전기 회로	단자들을 모두 연결하면서 전선의 길이가 가장 최소가 되도록 하는 문제
통 신	전화선의 길이가 최소가 되도록 전화 케이블 망을 구성하는 문제
배 관	파이프를 모두 연결하면서 파이프의 총 길이가 최소가 되도록 연결 하는 문제

❖ 최소 비용 신장 트리(Minimum Cost Spanning Tree)

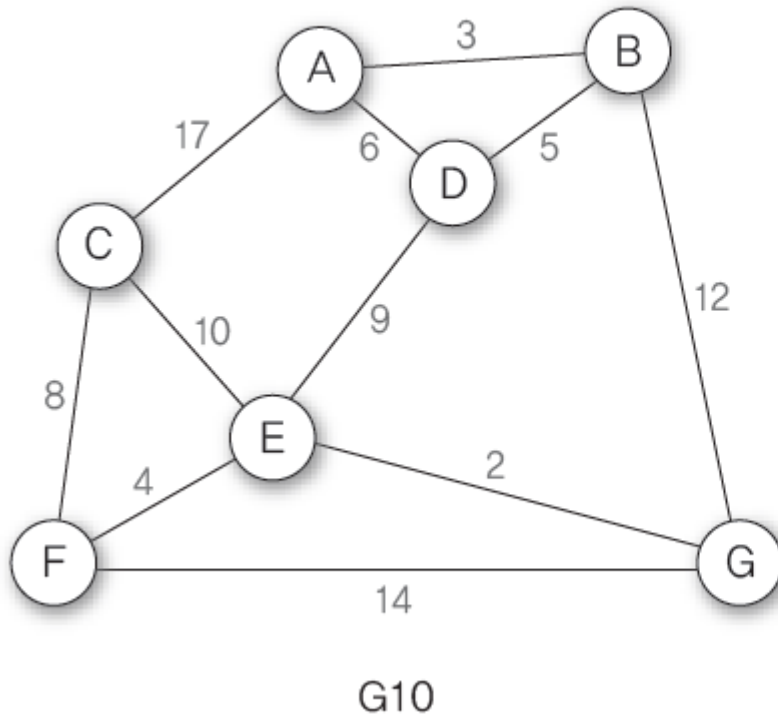
- 최소 비용 신장 트리를 만드는 알고리즘
 - 크루스칼(Kruskal)의 알고리즘
 - 프림(Prime)의 알고리즘
- 탐욕적인 방법(Greedy Method)
 - 알고리즘 설계에 있어서 중요한 기법 중 하나
 - 결정을 할 때 마다 그 순간 가장 좋다고 생각되는 것을 해답으로 선택함으로써 최종적인 해답에 도움
 - 탐욕적인 방법은 최적의 해답을 주는지를 반드시 검증해야 함
 - Kruskal 알고리즘은 최적의 해답을 주는 것으로 증명

❖ 크로스칼 알고리즘 I

- 가중치가 높은 간선을 제거하면서 최소 비용 신장 트리를 만드는 방법
- 알고리즘 순서
 - ① 그래프 G 의 모든 간선을 가중치에 따라 내림차순으로 정렬한다.
 - ② 그래프 G 에서 가중치가 가장 높은 간선을 제거한다. 단, 이때 정점을 그래프에서 분리시키는 간선은 제거할 수 없으므로 이런 경우에는 다음의 가중치가 높은 간선을 제거한다.
 - ③ 그래프 G 에 간선이 $n-1$ 개만 남을 때까지 ②를 반복한다.
 - ④ 그래프 간선이 $n-1$ 개만 남으면 최소 비용 신장 트리가 완성된다.

❖ 크로스칼 알고리즘 I

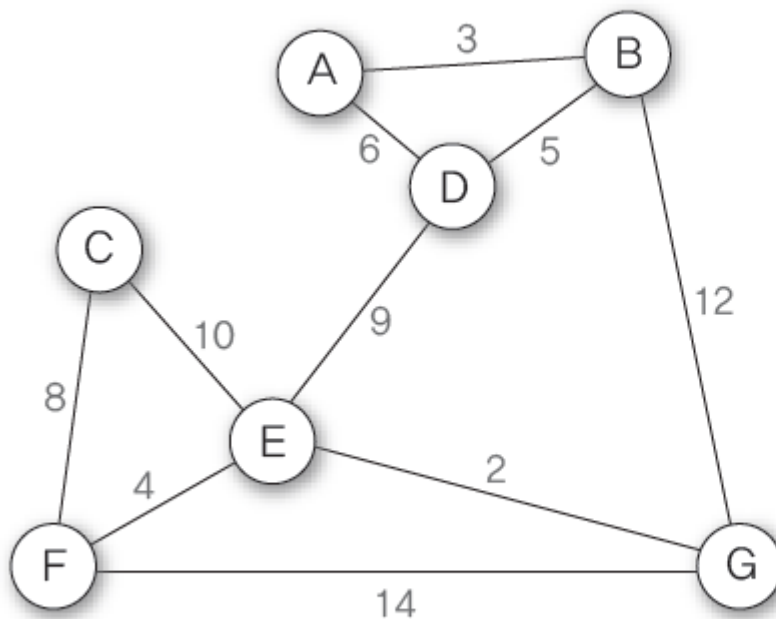
- 알고리즘을 이용하여 G10의 최소 비용 신장 트리 만들기
 - 초기 상태 : 그래프 G10의 간선을 가중치에 따라 내림차순 정렬



가중치	간선	간선 수 : 11개
17	(A, C)	
14	(F, G)	
12	(B, G)	
10	(C, E)	
9	(D, E)	
8	(C, F)	
6	(A, D)	
5	(B, D)	
4	(E, F)	
3	(A, B)	
2	(E, G)	

❖ 크로스칼 알고리즘 I

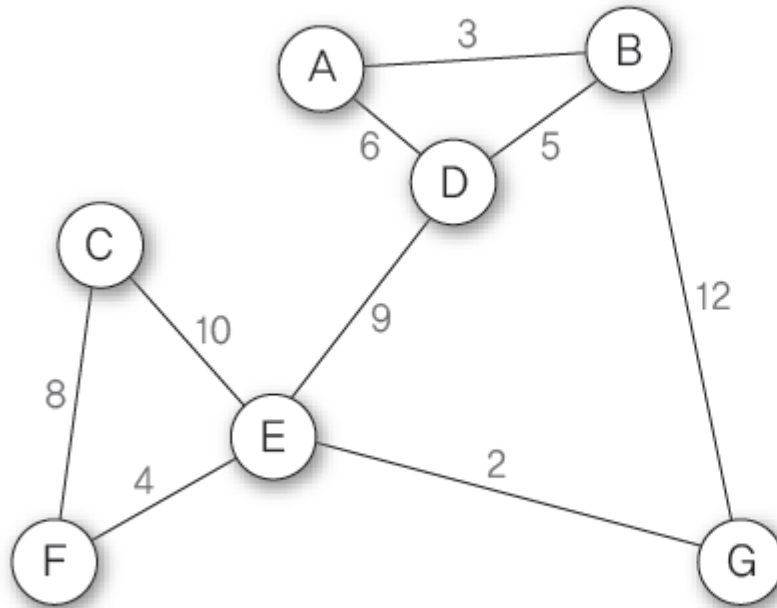
① 가중치가 가장 높은 간선(A, C)를 제거 → 남은 간선 수는 열 개



가중치	간선
17	(A, C)
14	(F, G)
12	(B, G)
10	(C, E)
9	(D, E)
8	(C, F)
6	(A, D)
5	(B, D)
4	(E, F)
3	(A, B)
2	(E, G)

❖ 크로스칼 알고리즘 I

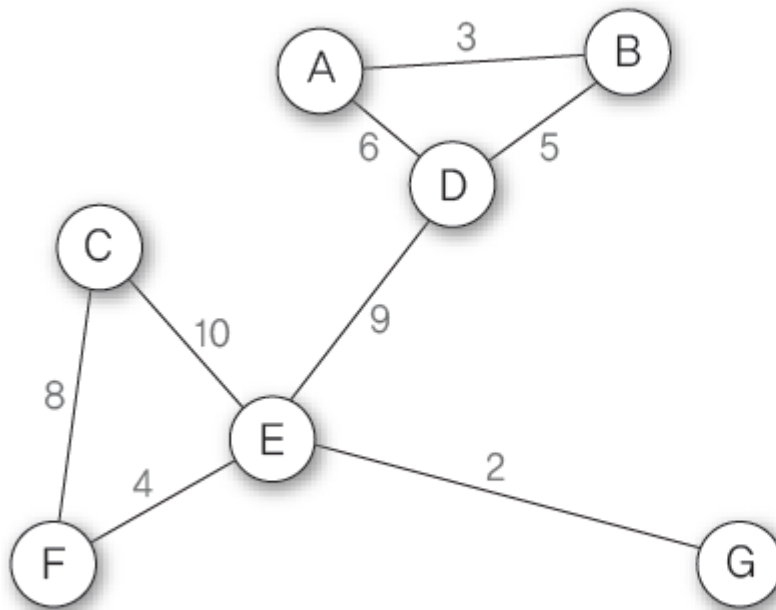
- ② 남은 간선 중에서 가중치가 가장 높은 간선(F, G)를 제거 → 남은 간선 수는 아홉 개



가중치	간선
17	(A, C)
14	(F, G)
12	(B, G)
10	(C, E)
9	(D, E)
8	(C, F)
6	(A, D)
5	(B, D)
4	(E, F)
3	(A, B)
2	(E, G)

❖ 크로스칼 알고리즘 I

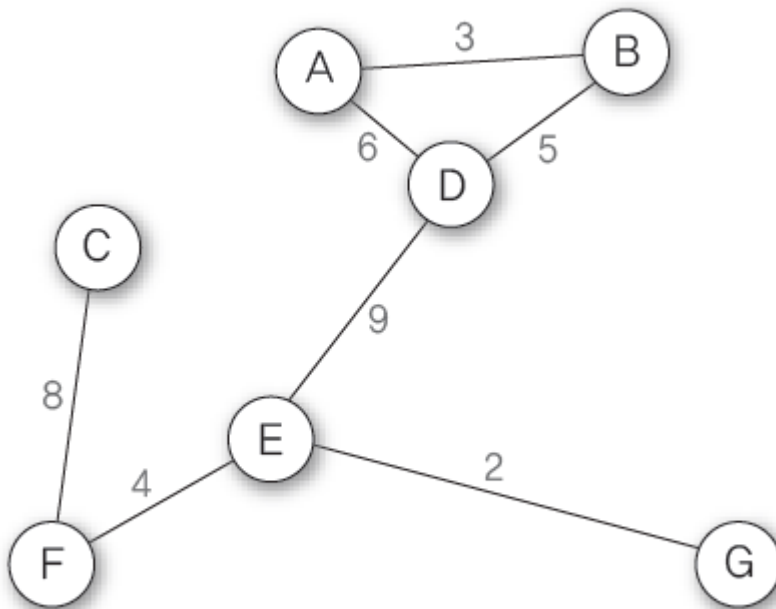
- ③ 남은 간선 중에서 가중치가 가장 높은 간선(B, G)를 제거 → 남은 간선 수는 여덟 개



가중치	간선
17	(A, C)
14	(F, G)
12	(B, G)
10	(C, E)
9	(D, E)
8	(C, F)
6	(A, D)
5	(B, D)
4	(E, F)
3	(A, B)
2	(E, G)

❖ 크로스칼 알고리즘 I

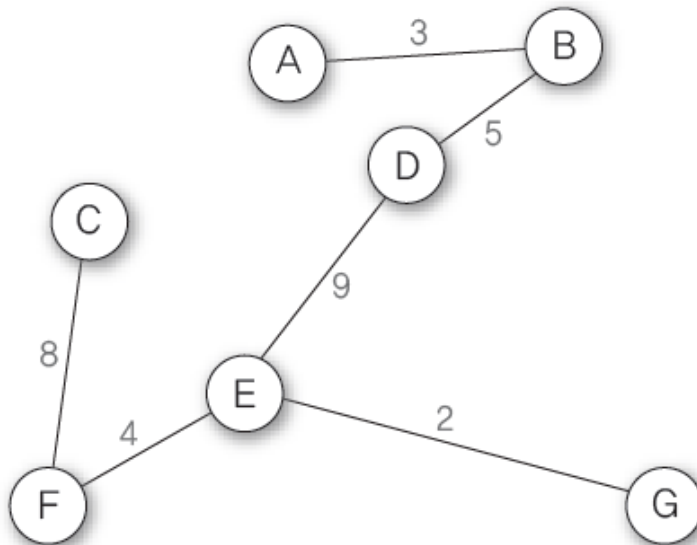
- ④ 남은 간선 중에서 가중치가 가장 높은 간선(C, E)를 제거 → 남은 간선 수는 일곱 개



가중치	간선
17	(A, C)
14	(F, G)
12	(B, G)
10	(C, E)
9	(D, E)
8	(C, F)
6	(A, D)
5	(B, D)
4	(E, F)
3	(A, B)
2	(E, G)

❖ 크로스칼 알고리즘 I

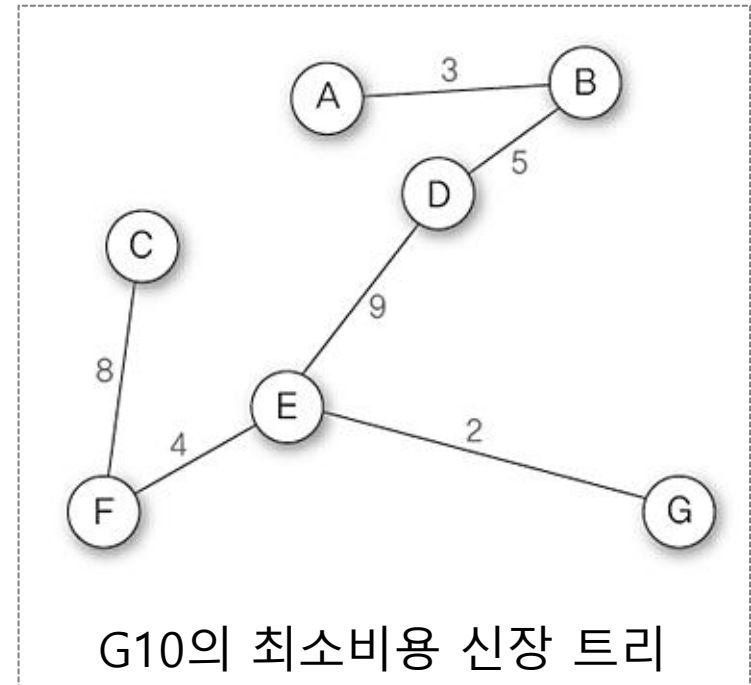
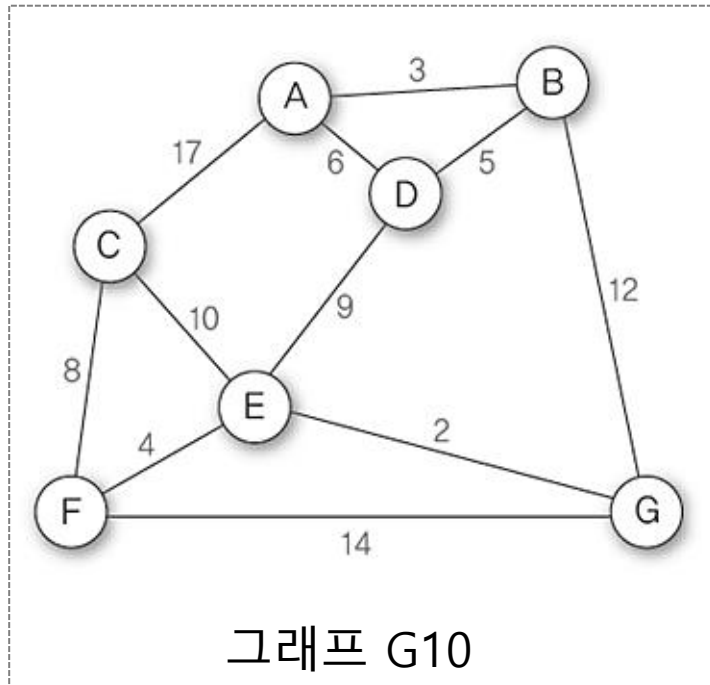
- ⑤ 남은 간선 중에서 가중치가 가장 높은 간선 (D, E)를 제거하면 그래프가 분리되어 단절 그래프가 되므로 그 다음으로 가중치가 높은 간선 (C, F)를 제거. 그런데 간선 (C, F)를 제거하면 정점 C가 그래프에서 분리되므로 제거 불가능. 따라서 그 다음으로 가중치가 높은 간선 (A, D)를 제거 → 남은 간선 수는 여섯 개



가중치	간선
17	(A, C)
14	(F, G)
12	(B, G)
10	(C, E)
9	(D, E)
8	(C, F)
6	(A, D)
5	(B, D)
4	(E, F)
3	(A, B)
2	(E, G)

❖ 크로스칼 알고리즘 I

- ⑥ 현재 남은 간선수가 여섯 개이므로 알고리즘 수행을 종료하면 신장 트리가 완성



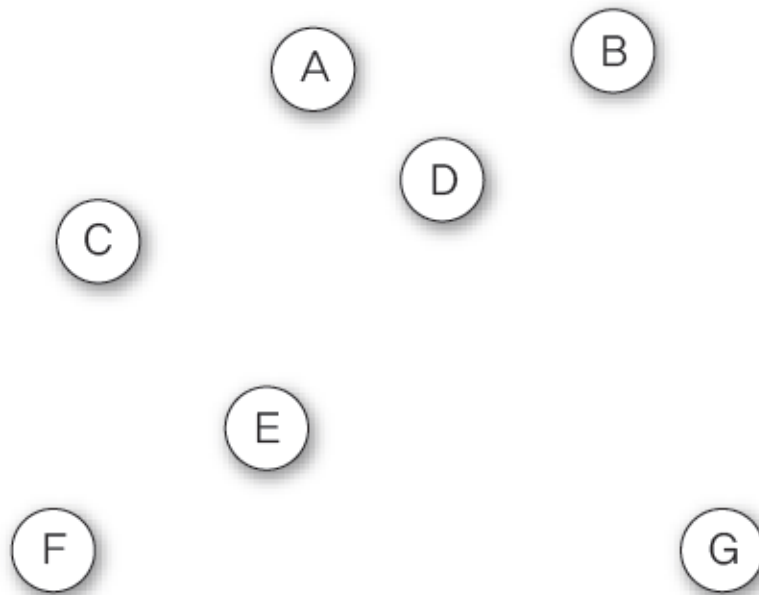
❖ 크로스칼 알고리즘 II

- 가중치가 낮은 간선을 삽입하면서 최소 비용 신장 트리를 만드는 방법
- 알고리즘 순서

- ① 그래프 G 의 모든 간선을 가중치에 따라 오름차순으로 정렬한다.
- ② 그래프 G 에서 가중치가 가장 낮은 간선을 삽입한다. 단, 이때 사이클을 형성하는 간선은 삽입할 수 없으므로 다음으로 가중치가 낮은 간선을 삽입한다.
- ③ 그래프 G 에 간선이 $n-1$ 개 삽입될 때까지 ②를 반복한다.
- ④ 그래프 간선이 $n-1$ 개가 되면 최소 비용 신장 트리가 완성된다.

❖ 크로스칼 알고리즘 II

- 알고리즘을 이용하여 G10의 최소 비용 신장 트리 만들기
 - 초기 상태 : 그래프 G10의 간선을 가중치에 따라 오름차순으로 정렬되어 있고 그래프 G10은 정점만 있음

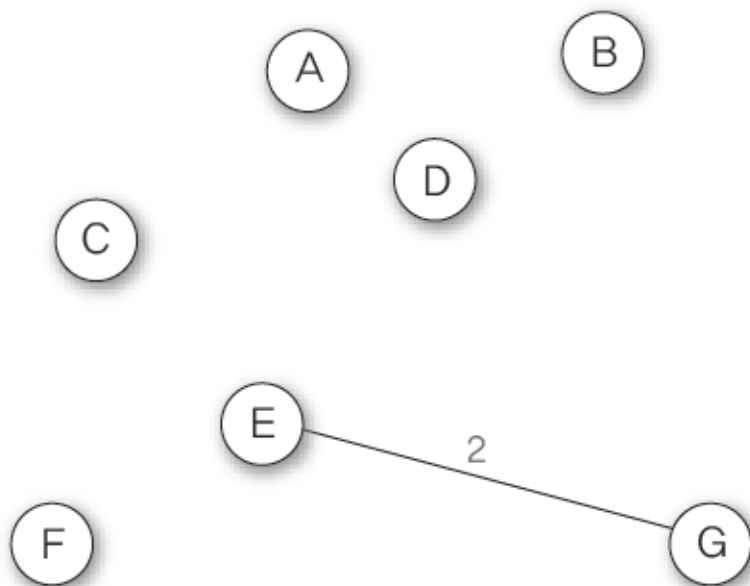


G10

가중치	간선	간선 수 : 11개
2	(E, G)	
3	(A, B)	
4	(E, F)	
5	(B, D)	
6	(A, D)	
8	(C, F)	
9	(D, E)	
10	(C, E)	
12	(B, G)	
14	(F, G)	
17	(A, C)	

❖ 크로스칼 알고리즘 II

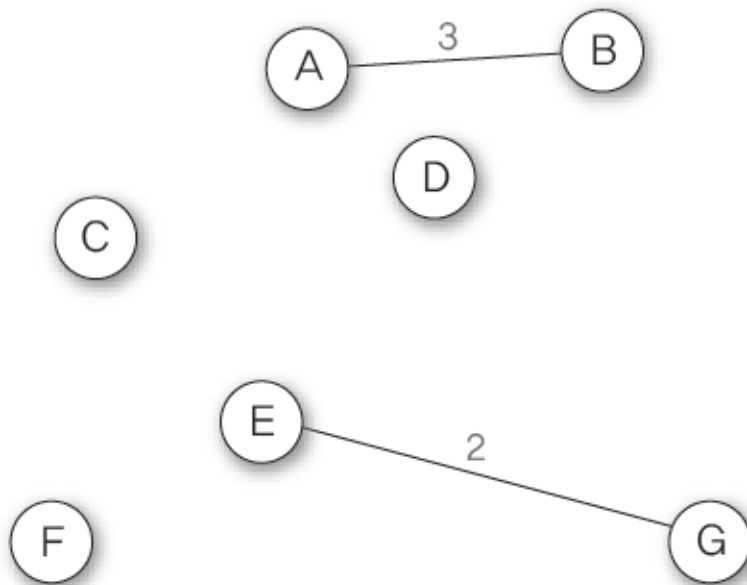
① 가중치가 가장 낮은 간선(E, G)를 삽입 → 삽입한 간선 수는 한 개



가중치	간선
2	(E, G)
3	(A, B)
4	(E, F)
5	(B, D)
6	(A, D)
8	(C, F)
9	(D, E)
10	(C, E)
12	(B, G)
14	(F, G)
17	(A, C)

❖ 크로스칼 알고리즘 II

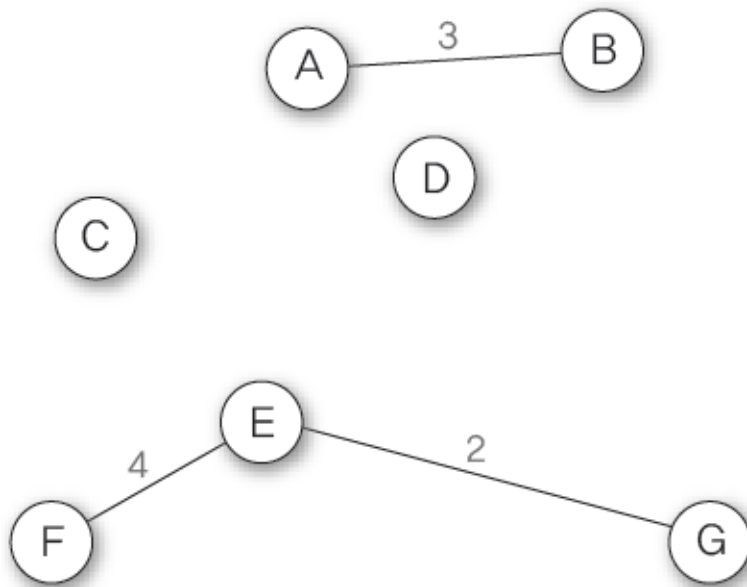
- ② 나머지 간선 중에서 가중치가 가장 낮은 간선(A, B)를 삽입 → 삽입한 간선 수는 두 개



가중치	간선
2	(E, G)
3	(A, B)
4	(E, F)
5	(B, D)
6	(A, D)
8	(C, F)
9	(D, E)
10	(C, E)
12	(B, G)
14	(F, G)
17	(A, C)

❖ 크로스칼 알고리즘 II

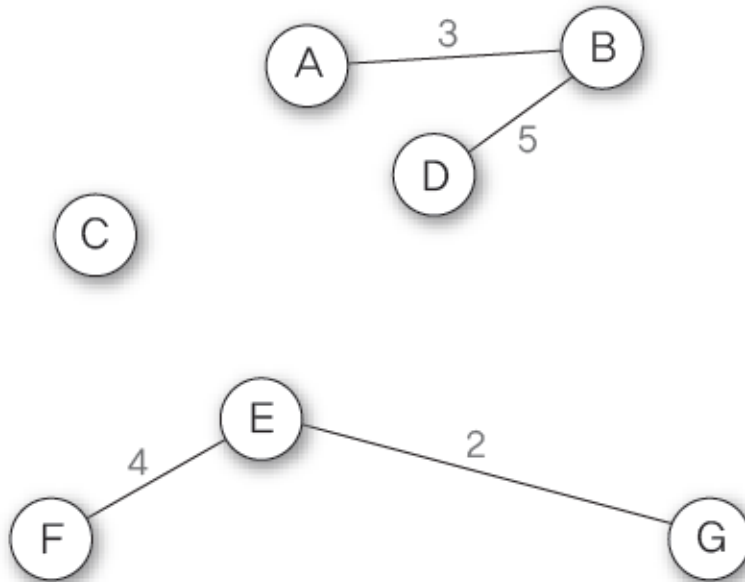
- ③ 나머지 간선 중에서 가중치가 가장 낮은 간선(E, F)를 삽입 → 삽입한 간선 수는 세 개



가중치	간선
2	(E, G)
3	(A, B)
4	(E, F)
5	(B, D)
6	(A, D)
8	(C, F)
9	(D, E)
10	(C, E)
12	(B, G)
14	(F, G)
17	(A, C)

❖ 크로스칼 알고리즘 II

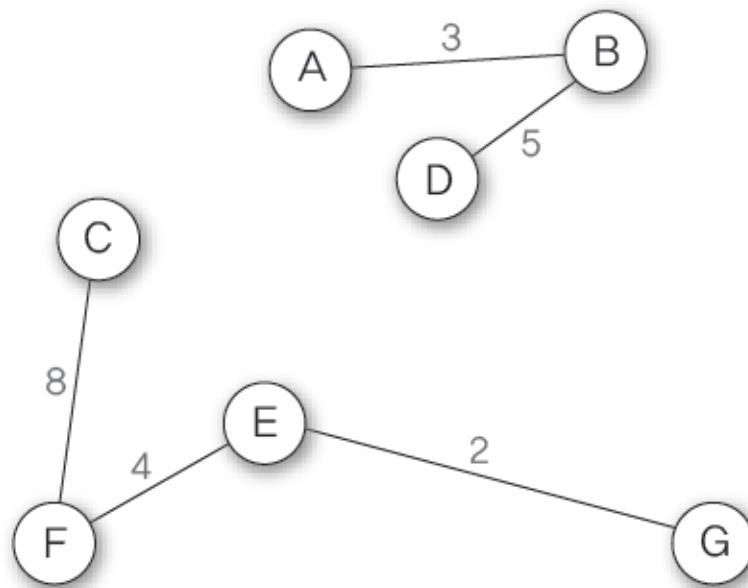
- ④ 나머지 간선 중에서 가중치가 가장 낮은 간선(B, D)를 삽입 → 삽입한 간선 수는 네 개



가중치	간선
2	(E, G)
3	(A, B)
4	(E, F)
5	(B, D)
6	(A, D)
8	(C, F)
9	(D, E)
10	(C, E)
12	(B, G)
14	(F, G)
17	(A, C)

❖ 크로스칼 알고리즘 II

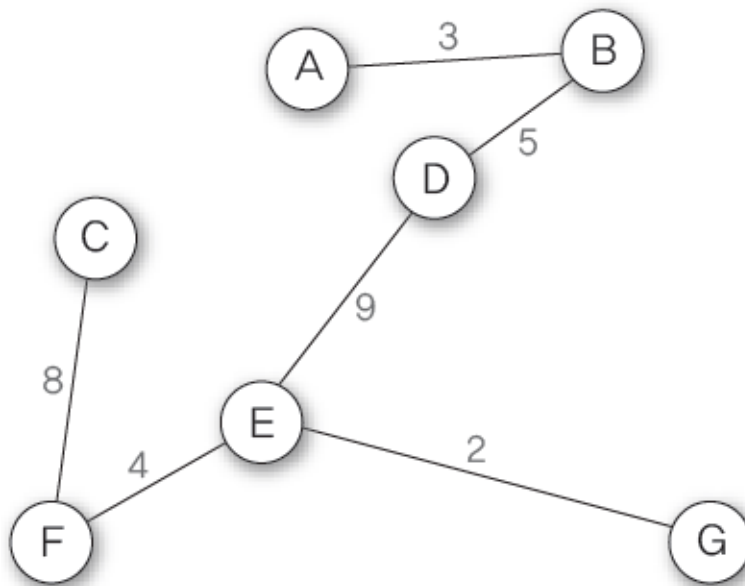
- ⑤ 나머지 간선 중에서 가중치가 가장 낮은 간선 (A, D)를 삽입하면 A-B-D-A의 사이클이 생성되므로 삽입할 수 없음 이런 경우에는 그 다음으로 가중치가 낮은 간선 (C, F)를 삽입 → 삽입한 간선 수는 다섯 개



가중치	간선
2	(E, G)
3	(A, B)
4	(E, F)
5	(B, D)
6	(A, D)
8	(C, F)
9	(D, E)
10	(C, E)
12	(B, G)
14	(F, G)
17	(A, C)

❖ 크로스칼 알고리즘 II

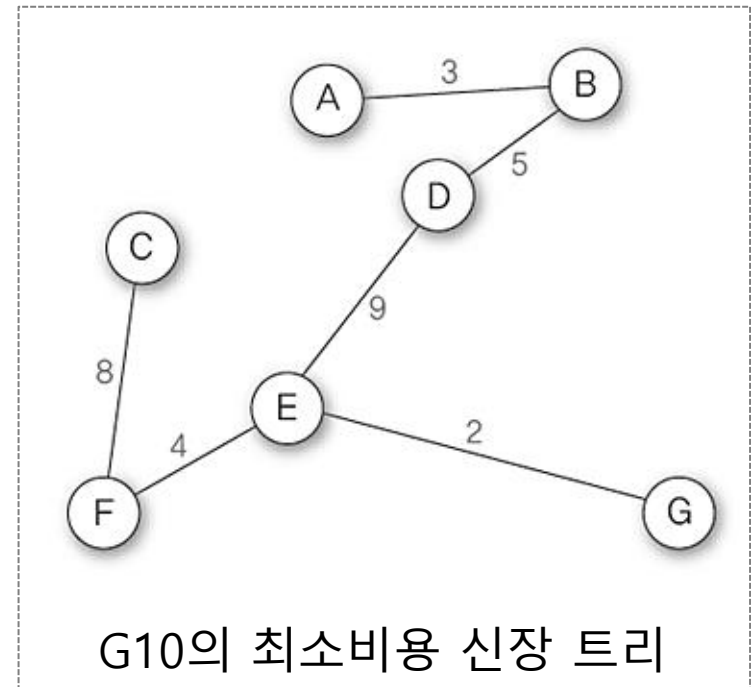
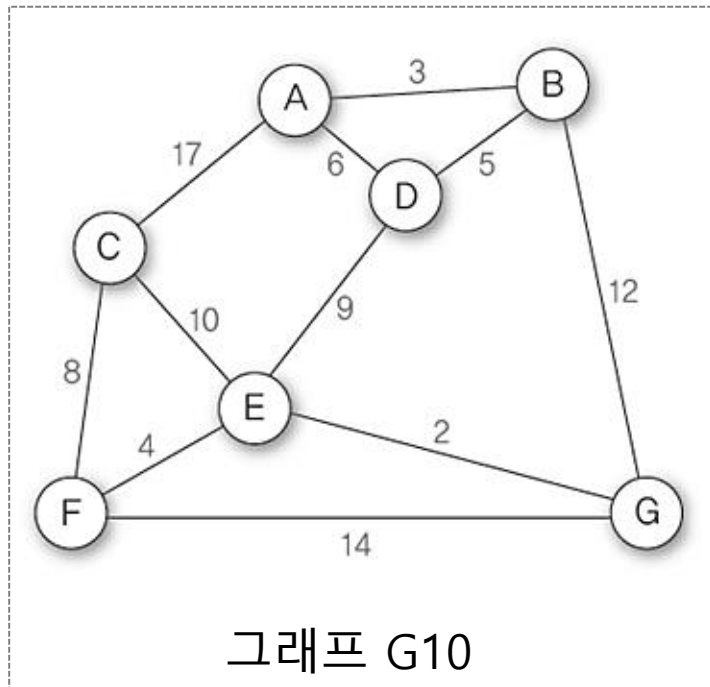
- ⑥ 나머지 간선 중에서 가중치가 가장 낮은 간선(D, E)를 삽입 → 삽입한 간선 수는 여섯 개



가중치	간선
2	(E, G)
3	(A, B)
4	(E, F)
5	(B, D)
6	(A, D)
8	(C, F)
9	(D, E)
10	(C, E)
12	(B, G)
14	(F, G)
17	(A, C)

❖ 크로스칼 알고리즘 II

- ⑦ 현재 삽입한 간선이 여섯 개이므로 알고리즘 수행을 종료하면 신장 트리 완성



❖ 크로스칼 알고리즘을 적용한 최소비용 신장 트리 프로그램

```
#include <stdio.h>
#include <stdlib.h>
#define TRUE 1
#define FALSE 0
#define MAX_VERTICES 100
#define INF 1000

int parent[MAX_VERTICES]; // 부모 노드

// 간선을 나타내는 구조체
struct Edge
{
    int start, end, weight;
};
// 그래프 구조
typedef struct GraphType
{
    int n;
    struct Edge edges[2 * MAX_VERTICES];
} GraphType;
```

❖ 크로스칼 알고리즘을 적용한 최소비용 신장 트리 프로그램

```
// 부모 노드의 초기화
void set_init(int n)
{
    int i;
    for (i = 0; i < n; i++)
        parent[i] = -1;
}

// 집합을 반환
int set_find(int curr)
{
    if (parent[curr] == -1)
        return curr;           // 루트
    while (parent[curr] != -1) curr = parent[curr];
    return curr;
}
```

❖ 크로스칼 알고리즘을 적용한 최소비용 신장 트리 프로그램

```
// 두개의 원소가 속한 집합을 합친다.
void set_union(int a, int b)
{
    int root1 = set_find(a); // 노드 a의 루트를 찾는다.
    int root2 = set_find(b); // 노드 b의 루트를 찾는다.
    if (root1 != root2) // 합한다.
        parent[root1] = root2;
}

// 그래프 초기화
void graph_init(GraphType* g)
{
    int i;
    g->n = 0;
    for (i = 0; i < 2 * MAX_VERTICES; i++) {
        g->edges[i].start = 0;
        g->edges[i].end = 0;
        g->edges[i].weight = INF;
    }
}
```

❖ 크로스칼 알고리즘을 적용한 최소비용 신장 트리 프로그램

```
// 간선 삽입 연산
void insert_edge(GraphType* g, int start, int end, int w)
{
    g->edges[g->n].start = start;
    g->edges[g->n].end = end;
    g->edges[g->n].weight = w;
    g->n++;
}

// qsort()에 사용되는 함수
int compare(const void* a, const void* b)
{
    struct Edge* x = (struct Edge*)a;
    struct Edge* y = (struct Edge*)b;
    return (x->weight - y->weight);
}
```

❖ 크로스칼 알고리즘을 적용한 최소비용 신장 트리 프로그램

```
// kruskal의 최소 비용 신장 트리 프로그램
void kruskal(GraphType *g)
{
    int edge_accepted = 0;    // 현재까지 선택된 간선의 수
    int uset, vset;           // 정점 u와 정점 v의 집합 번호
    struct Edge e;

    set_init(g->n);           // 집합 초기화
    qsort(g->edges, g->n, sizeof(struct Edge), compare);
```


❖ 크로스칼 알고리즘을 적용한 최소비용 신장 트리 프로그램

```
printf("크루스칼 최소 신장 트리 알고리즘 %n");
int i = 0;
while (edge_accepted < (g->n - 1))    // 간선의 수 < (n-1)
{
    e = g->edges[i];
    uset = set_find(e.start);    // 정점 u의 집합 번호
    vset = set_find(e.end);    // 정점 v의 집합 번호
    if (uset != vset) {        // 서로 속한 집합이 다르면
        printf("간선 (%c,%c) %d 선택\n", (e.start+65), (e.end+65), e.weight);
        edge_accepted++;
        set_union(uset, vset);    // 두개의 집합을 합친다.
    }
    i++;
}
}
```

❖ 크로스칼 알고리즘을 적용한 최소비용 신장 트리 프로그램

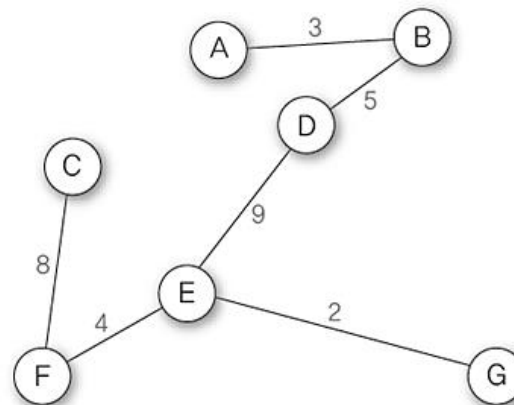
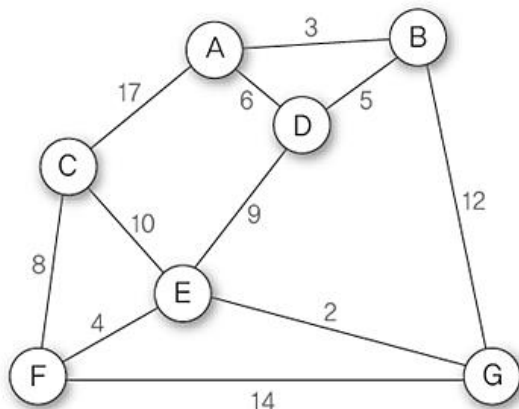
```
int main()
{
    GraphType *G10;
    G10 = (GraphType *)malloc(sizeof(GraphType));
    graph_init(G10);
    insert_edge(G10, 0, 1, 3);
    insert_edge(G10, 0, 2, 17);
    insert_edge(G10, 0, 3, 6);
    insert_edge(G10, 1, 6, 12);
    insert_edge(G10, 1, 3, 5);
    insert_edge(G10, 2, 4, 10);
    insert_edge(G10, 2, 5, 8);
    insert_edge(G10, 3, 4, 9);
    insert_edge(G10, 4, 5, 4);
    insert_edge(G10, 4, 6, 2);
    insert_edge(G10, 5, 6, 14);
    kruskal(G10);
    free(G10);
    return 0;
}
```

신장 트리와 최소 비용 신장 트리

❖ 크로스칼 알고리즘을 적용한 최소비용 신장 트리 프로그램

■ 실행 화면

```
C:\WINDOWS\system32\cmd.exe
크루스칼 최소 신장 트리 알고리즘
간선 (E, G) 2 선택
간선 (A, B) 3 선택
간선 (E, F) 4 선택
간선 (B, D) 5 선택
간선 (C, F) 8 선택
간선 (D, E) 9 선택
계속하려면 아무 키나 누르십시오 . . .
```



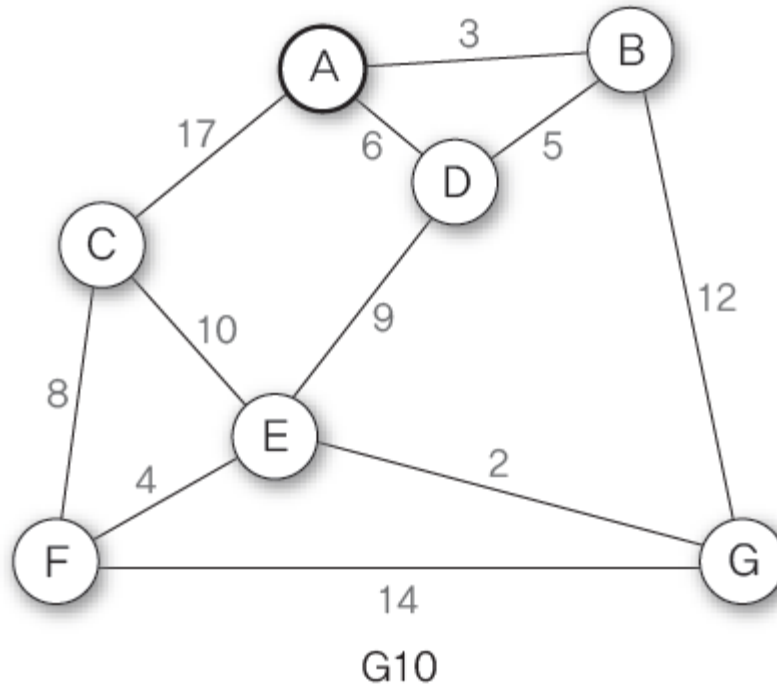
❖ 프림(Prime) 알고리즘

- 간선을 정렬하지 않고 하나의 정점에서 시작하여 트리를 확장해 나가는 방법
- 알고리즘 순서

- ① 그래프 G 에서 시작 정점을 선택한다.
- ② 선택한 정점에 부속된 모든 간선 중에서 가중치가 가장 낮은 간선을 연결하여 트리를 확장한다.
- ③ 이전에 선택한 정점과 새로 확장된 정점에서 부속된 모든 간선 중에서 가중치가 가장 낮은 간선을 삽입한다. 단, 사이클을 형성하는 간선은 삽입할 수 없으므로 이런 경우에는 그 다음으로 가중치가 낮은 간선을 선택한다.
- ④ 그래프 G 에 간선이 $n-1$ 개 삽입될 때까지 ③을 반복한다.
- ⑤ 그래프 G 의 간선이 $n-1$ 개가 되면 최소 비용 신장 트리가 완성 된다.

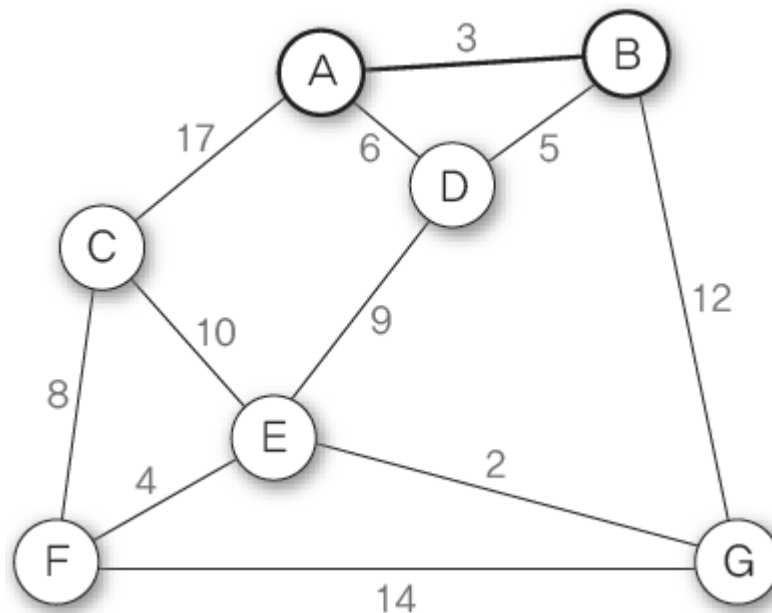
❖ 프림(Prime) 알고리즘

- 알고리즘을 이용하여 그래프 G10의 최소 비용 신장 트리 만들기
 - 초기 상태 : 그래프 G10의 정점 중에서 정점 A를 시작 정점으로 선택



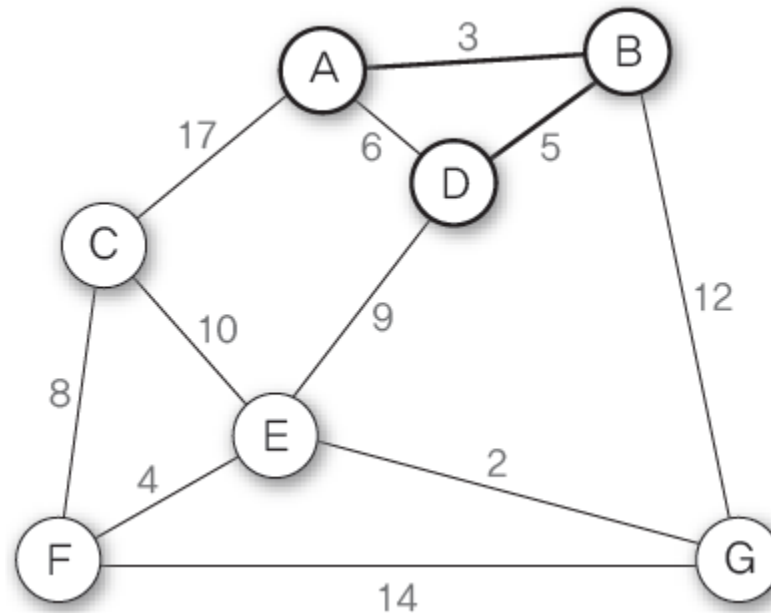
❖ 프림(Prime) 알고리즘

- ① 정점 A에 소속된 간선 중에서 가중치가 가장 낮은 간선 (A, B)를 삽입하여 트리를 확장→ 삽입한 간선 수는 한 개



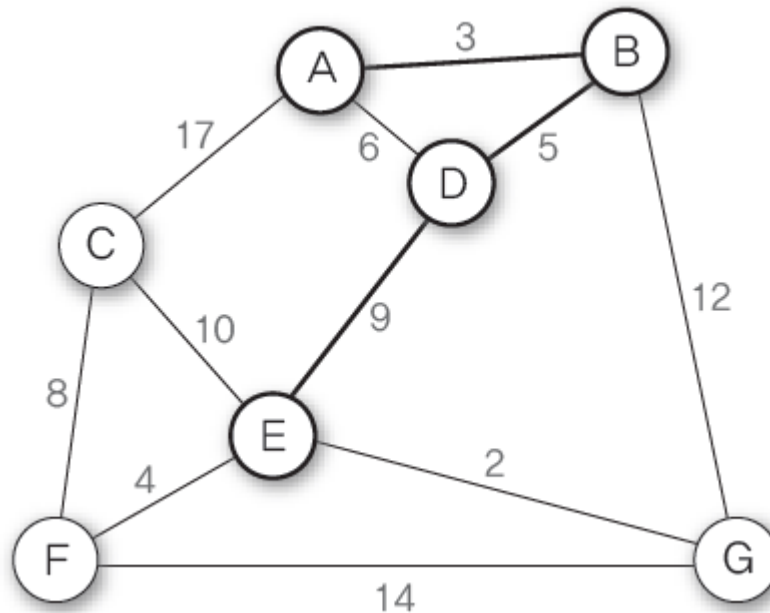
❖ 프림(Prime) 알고리즘

- ② 현재 확장된 트리의 정점 A, B에 소속된 간선 중에서 가중치가 가장 낮은 간선 (B, D)를 삽입하여 트리를 확장→ 삽입한 간선 수는 두 개



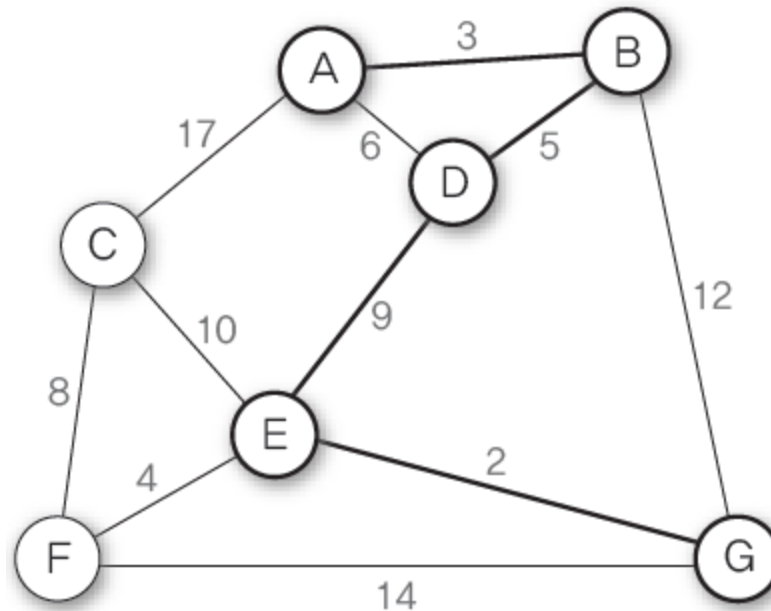
❖ 프림(Prime) 알고리즘

- ③ 현재 확장된 트리의 정점 A, B, D에 소속된 간선 중에서 가중치가 가장 낮은 간선 (A, D)를 삽입하면 A-B-D-A의 사이클이 생성되므로 삽입할 수 없음 따라서 그 다음으로 가중치가 낮은 간선 (D, E)를 삽입 → 삽입한 간선 수는 세 개, 삽입 불가능한 간선은 (A, D)



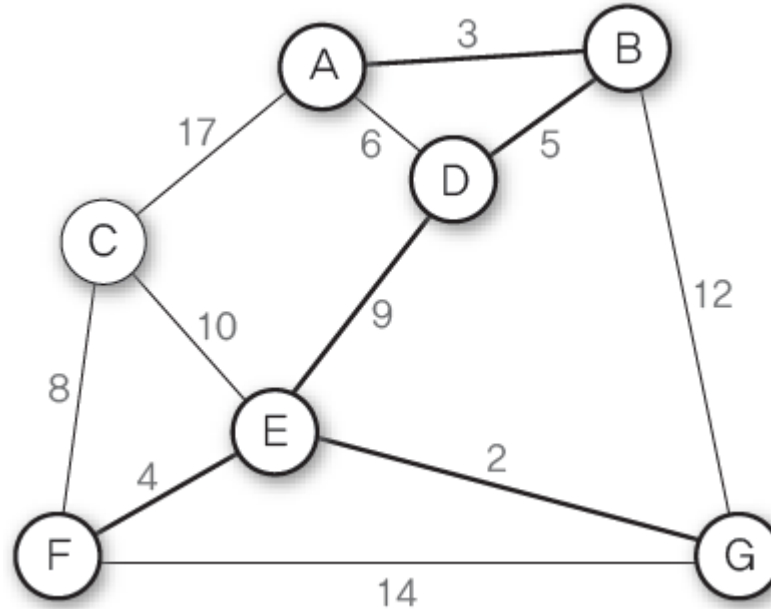
❖ 프림(Prime) 알고리즘

- ④ 현재 확장된 트리의 정점 A, B, D, E에 소속된 간선 중에서 가중치가 가장 낮은 간선 (E, G)를 삽입하여 트리를 확장→ 삽입한 간선 수는 네 개, 삽입 불가능한 간선은(A, D)



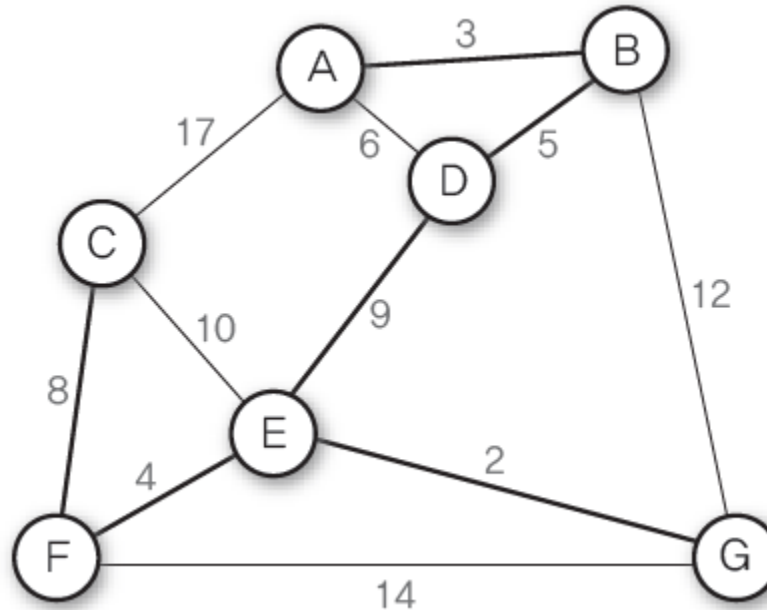
❖ 프림(Prime) 알고리즘

- ⑤ 현재 확장된 트리의 정점 A, B, D, E, G에 소속된 간선 중에서 가중치가 가장 낮은 간선 (E, F)를 삽입하여 트리를 확장→ 삽입한 간선 수는 다섯 개 , 삽입 불가능한 간선은(A, D)



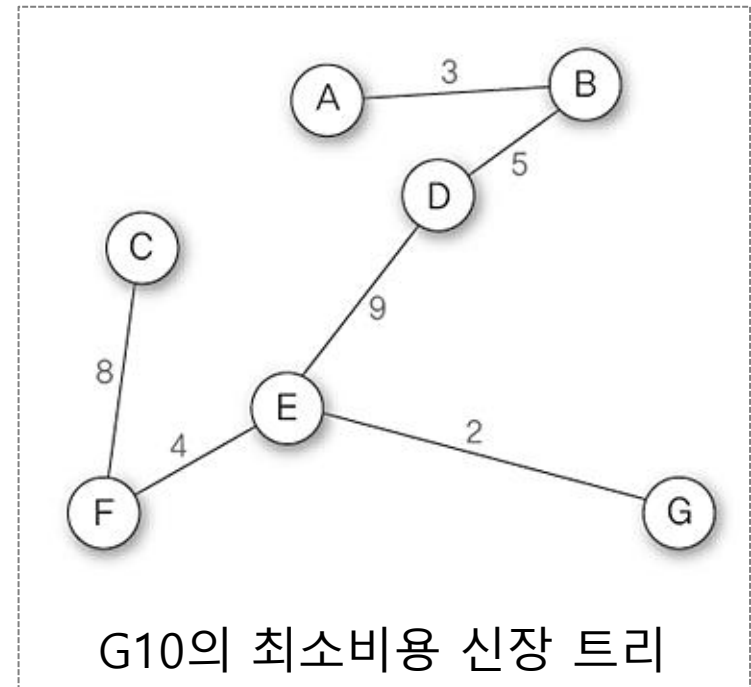
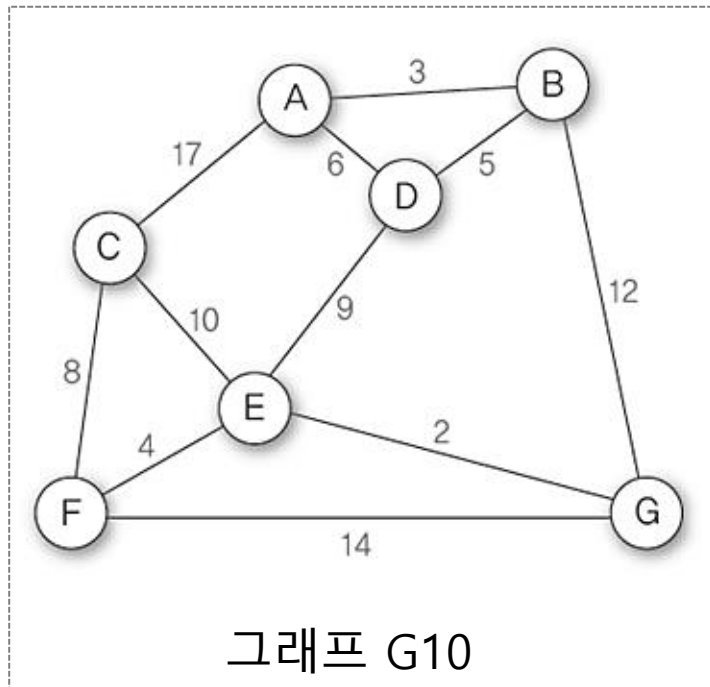
❖ 프림(Prime) 알고리즘

- ⑥ 현재 확장된 트리의 정점 A, B, D, E, G에 소속된 간선 중에서 가중치가 가장 낮은 간선 (C, F)를 삽입하여 트리를 확장→ 삽입한 간선 수는 여섯 개, 삽입 불가능한 간선은(A, D)



❖ 프림(Prime) 알고리즘

- ⑦ 현재 삽입한 간선 수가 여섯 개이므로 알고리즘 수행을 종료하면 신장 트리 완성



❖ 프림(Prime) 알고리즘을 적용한 최소신장 트리 프로그램 예제

```
#include <stdio.h>
#include <stdlib.h>

#define TRUE 1
#define FALSE 0
#define MAX_VERTICES 100
#define INF 1000L

typedef struct GraphType {
    int n;    // 정점의 개수
    int weight[MAX_VERTICES][MAX_VERTICES];
} GraphType;

int selected[MAX_VERTICES];
int distance[MAX_VERTICES];
```

❖ 프림(Prime) 알고리즘을 적용한 최소신장 트리 프로그램 예제

```
// 최소 dist[v] 값을 갖는 정점을 반환
int get_min_vertex(int n)
{
    int v, i;
    for (i = 0; i < n; i++)
        if (!selected[i]) {
            v = i;
            break;
        }
    for (i = 0; i < n; i++)
        if (!selected[i] && (distance[i] < distance[v])) v = i;
    return (v);
}
```

❖ 프림(Prime) 알고리즘을 적용한 최소신장 트리 프로그램 예제

```
void prim(GraphType* g, int s)
{
    int i, u, v;

    for (u = 0; u < g->n; u++)
        distance[u] = INF;
    distance[s] = 0;
    for (i = 0; i < g->n; i++)
    {
        u = get_min_vertex(g->n);
        selected[u] = TRUE;
        if (distance[u] == INF)
            return;
        printf("정점 %C 추가\n", u+65);
        for (v = 0; v < g->n; v++)
            if (g->weight[u][v] != INF)
                if (!selected[v] && g->weight[u][v] < distance[v])
                    distance[v] = g->weight[u][v];
    }
}
```

❖ 프림(Prime) 알고리즘을 적용한 최소신장 트리 프로그램 예제

```
int main(void)
{
    GraphType g = { 7,
        { 0, 3, 17, 6, INF, INF, INF },
        { 3, 0, INF, 5, INF, INF, 12 },
        { 17, INF, 0, INF, 10, 8, INF },
        { 6, 5, INF, 0, 9, INF, INF },
        { INF, INF, 10, 9, 0, 4, 2 },
        { INF, INF, 8, INF, 4, 0, 14 },
        { INF, 12, INF, INF, 2, 14, 0 } }
    };
    prim(&g, 0);
    return 0;
}
```


신장 트리와 최소 비용 신장 트리

❖ 프림(Prime) 알고리즘을 적용한 최소신장 트리 프로그램 예제

■ 실행 화면

```
C:\WINDOWS\system32\cmd.exe
정점 A 추가
정점 B 추가
정점 D 추가
정점 E 추가
정점 G 추가
정점 F 추가
정점 C 추가
계속하려면 아무 키나 누르십시오 . . .
```

